CostPilot — Phase 1 Plan

Management Accounting Copilot for FP&A and Cost Teams

November 2026

- 1. Executive Summary
- 2. Problems Being Solved
- 3. Scope
  - 3.1 In scope for Phase 1
  - 3.2 Deliberately out of scope for Phase 1
- 4. Architecture
- 5. Technology Stack (Phase 1)
- 6. Delivery Plan
- 7. Success Criteria
- 8. Key Risks and Mitigations

1. Executive Summary

CostPilot is an AI copilot for management accounting. Phase 1 targets the recurring, high-cost work of FP&A and cost-accounting teams in large enterprises: variance explanation, budget-versus-actual analysis, and short-run economic decisions (make or buy, special orders, product discontinuation).

The defining design rule: **the language model never performs arithmetic.** A deterministic calculation engine computes every number; the model only interprets the user's question, extracts parameters, and narrates the result. Every answer carries a full audit trail — source data, assumptions, and calculation chain — because explainability, not intelligence, is what earns adoption in a finance organization.

Phase 1 outcome: a working copilot that answers cost questions and runs decision analyses over a realistic manufacturing cost dataset, with auditable outputs a CFO can defend.

2. Problems Being Solved

Pain points observed in large industries, in priority order:

#	Pain	Current cost	Phase 1 module
1	Month-end close takes 8–12 days; controllers hand-write flux justifications for every account movement	Hundreds of analyst-hours per close cycle	Variance & Flux Explainer
2	Variance analysis arrives weeks after the event; price/usage/rate/efficiency decomposition done	Late corrective action; blame-driven review culture	Variance & Flux Explainer

#	Pain	Current cost	Phase 1 module
	manually in error-prone spreadsheets		
3	Short-run decisions (special orders, make/buy, add/drop) are delayed for days of data pulls, so they are made on intuition — orders accepted below true incremental cost	Direct margin leakage per decision	Short-Run Decision Engine
4	Fixed-versus-variable classification of accounts is inconsistent or undocumented, corrupting every downstream analysis	Hidden; distorts CVP, budgeting, costing globally	Cost Data Layer
5	Management P&L and statutory P&L maintained in parallel and never reconciled cleanly	Reconciliation effort every period	Cost Data Layer + explainable bridges

Static budgeting distorting performance evaluation (the Caterpillar problem) is addressed inside Module 2 by making flexible-budget comparison the default: volume effects are separated from controllable performance in every variance output.

3. Scope

3.1 In scope for Phase 1

Module 0 — Cost Data Layer (P0)

- Ingestion of Excel/CSV templates: general-ledger extracts, standard cost cards, budgets, production and sales volumes.
- Account registry: every cost account tagged fixed / variable / semi-variable (with estimation method) and direct / indirect, with entity, plant, product, and period dimensions.
- Deterministic storage: DuckDB (v1) with a documented schema that can migrate to Postgres.
- Data quality gates on load: completeness of tagging, reconciliation checks, out-of-range detection.

Module 1 — Variance & Flux Explainer (P1)

- Material variances: MCV = MPV + MUV (MPV = AQ × (SP − AP); MUV = SP × (SQ − AQ)).
- Labour variances: LCV = LRV + LEV (LRV = AH × (SR − AR); LEV = SR × (SH − AH)).
- Overhead variances: spending and volume/capacity splits.
- Flexible-budget restatement before comparison (volume effect isolated from performance effect).
- Budget-versus-actual flux analysis at account level with drill-down to the transactions behind each movement.
- Narrative draft for each variance: magnitude, decomposition, likely causes, owner follow-ups — Favourable/Adverse labelled, management-by-exception thresholds applied so only significant variances escalate.

Module 2 — Short-Run Decision Engine (P2)

Seven guided decision workflows, each producing a differential-analysis table plus a recommendation:

1. Make or buy — relevant-cost comparison, unavoidable fixed overhead excluded, capacity and supplier-risk checklist.
2. Special order — contribution check against idle capacity, regular-sales cannibalization and price-precedent flags.
3. Add or drop product — contribution lost versus avoidable fixed costs saved; explicit warning against dropping a loss-reporting product that still covers avoidable costs.
4. Sell or process further — incremental revenue versus incremental processing cost; joint costs treated as sunk and excluded.
5. Operate or shutdown — contribution versus avoidable fixed costs for the shutdown window.
6. Product mix under a limiting factor — ranking by contribution per unit of the scarce resource.
7. Break-even / target-profit (CVP) — BEP units = Fixed Cost ÷ Contribution per unit, margin of safety, and "what-if" changes to price, volume, cost.

Module 3 — Explainability Shell (cross-cutting)

Every response includes: the numbers used and their source records, the accounting rules applied (relevant versus sunk cost, avoidable versus unavoidable, flexible-budget basis), the full calculation chain, and stated assumptions the user can correct before the answer finalizes.

Costing method support: both variable (contribution-format) and absorption (gross-profit-format) income statements, with an automatic reconciliation bridge: profit difference = fixed manufacturing overhead deferred in or released from inventory.

3.2 Deliberately out of scope for Phase 1

Excluded	Reason	Where it goes
ABC / TDABC product profitability	Needs activity and driver transaction data most companies cannot supply quickly	Phase 2
Budgeting and forecasting workflows	Organizational process change more than software	Phase 2
Performance measurement (Balanced Scorecard, ROI vs Residual Income)	Valuable but secondary to the close-pain wedge	Phase 2
ERP connectors (SAP, Oracle, NetSuite)	Integration effort is large; v1 input via templates proves the concept	Phase 2
Marketing decision analytics (channel mix, CAC/LTV, pricing elasticity)	No domain knowledge layer exists yet; must be built	Phase 3
Multi-currency, intercompany, consolidation	Enterprise rollout concern, not concept proof	Phase 3

4. Architecture

```
User question (chat or guided workflow form)
        |
        v
[LLM: intent classification + parameter extraction]   -- no arithmetic
        |
        v
[Cost Data Layer]  account registry, dimensions, validated loads
        |
        v
[Deterministic Engine]  Python + SQL (DuckDB): variance formulas,
        |               CVP math, differential analysis, flexible budgets
        v
[LLM: narrative generation + table rendering]
        |
        v
[Response + Audit Trail]  sources, assumptions, calculation chain
```

Design constraints:

- **Repeatability:** the same question on the same data must return the same number, byte for byte. The engine is pure computation; the model only wraps it.
- **Rule fidelity:** engine formulas map 1:1 to the standard management-accounting rule set (cost classification, standard costing, CVP, differential analysis), so outputs are verifiable against textbook solutions.
- **Human-in-the-loop:** parameter extraction is echoed back for confirmation before calculation; the user can edit any extracted value.
- **No silent assumptions:** when data is missing (e.g., avoidable fixed cost not tagged), the copilot asks rather than defaults.

5. Technology Stack (Phase 1)

Layer	Choice	Rationale
API / orchestration	Python, FastAPI	Async, OpenAPI out of the box, team familiarity assumed
Analytical store	DuckDB	Zero-ops columnar engine, ideal for variance-scale data; Postgres migration path kept
Calculation engine	Plain Python + SQL modules	Auditable, unit-testable, no framework lock-in
LLM	Frontier model via API with function calling only	Model plans and narrates; tools compute
Ingress	Excel/CSV templates	Real-world input format of finance teams; no ERP dependency
Frontend	Chat UI with structured analysis cards (tables, variance waterfalls)	Numbers must render as finance tables, not prose
Tests	Golden-case suite: every worked example from the reference	Formula correctness gate before any UX polish

Layer	Choice	Rationale
	curriculum must be reproduced exactly	

6. Delivery Plan

Milestone	Contents	Exit criteria
M1 — Data layer + golden formulas	Schema, account registry, CSV ingestion, engine with all Unit-7/8 formulas, test suite	All golden cases pass; a 3-product / 2-plant synthetic dataset loads and reconciles
M2 — Variance explainer	Flexible-budget restatement, material/labour/overhead variance decomposition, flux drafts	Upload a month of actuals, receive a close-ready variance pack with narratives
M3 — Decision engine	Seven guided workflows with parameter confirmation	"Special order of 2,000 units at ₹380 with idle capacity" returns the full differential analysis in under 10 seconds
M4 — Chat shell + audit trail	NL-to-workflow routing, response cards, audit-trail panel	Non-accountant can obtain a correct, sourced answer unassisted
M5 — Pilot demo	Packaging, documentation, one live-data trial with a friendly company	Pilot user reproduces one real close-cycle variance pack end to end

Suggested sequencing detail: build M1 and the golden-test suite first; the demo that sells Phase 1 is M3's special-order answer running end to end.

7. Success Criteria

- **Correctness:** 100% match on golden-case tests; zero LLM-generated numbers in any output.
- **Speed:** decision-analysis answer in under 10 seconds; full monthly variance pack in under one minute.
- **Adoption proxy:** a finance user can complete a flux explanation without writing a spreadsheet formula.
- **Trust:** every response's audit trail allows an accountant to re-perform the calculation manually from the cited figures.

8. Key Risks and Mitigations

Risk	Mitigation
Cost data arrives untagged (fixed/variable unknown) — the classic killer	Registry is a first-class deliverable with quality gates; semi-variable estimation documented; pilot chosen for data readiness

Risk	Mitigation
LLM silently invents a number	Architectural: arithmetic is unreachable by the model; output validator rejects responses containing figures not produced by the engine
Recommendations trusted without scrutiny	Explainability shell is mandatory UI, not a debug view; every recommendation states assumptions needing confirmation
Textbook rules fail for real industries (e.g., service-sector cost structures)	Phase 1 anchors on discrete manufacturing; vertical expansion after pilot
Scope creep toward ERP integration before concept proof	Out-of-scope table treated as a contract; ERP work starts only after M5 pilot feedback